

543. Diameter of Binary Tree Easy

Acceptance: 64.9% | [LeetCode](#)

Given the `root` of a binary tree, return *the length of the diameter of the tree*.

The **diameter** of a binary tree is the **length** of the longest path between any two nodes in a tree. This path may or may not pass through the `root`.

The **length** of a path between two nodes is represented by the number of edges between them.

Example 1:

```
**Input:** root = [1,2,3,4,5]
**Output:** 3
```

****Explanation:**** 3 is the length of the path [4,2,1,3] or [5,2,1,3].

Example 2:

```
**Input:** root = [1,2]
**Output:** 1
```

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- `-100 <= Node.val <= 100`

Claude Opus Solution

Diameter of Binary Tree

1. Intuition

The **key insight** is recognizing what a "path through a node" actually means:

For any node, the longest path that **passes through it** equals `(height of left subtree) + (height of right subtree)`.

The diameter of the whole tree is simply the **maximum** of this value over **every** node in the tree.

The clever trick: while computing the **height** of each subtree (a standard DFS), we can *simultaneously* update a running maximum diameter. This lets us solve everything in a single traversal instead of computing height repeatedly for each node (which would be $O(n^2)$).

This is the classic **"compute one thing, track another as a side effect"** recursion pattern.

2. Approach (Pseudocode)

```
diameter = 0 (global tracker)

function height(node):
    if node is null:
        return 0 # empty subtree has height 0 (in edge count)

    left_height = height(node.left) # edges below on the left
    right_height = height(node.right) # edges below on the right

    # Path through THIS node = left edges + right edges
    diameter = max(diameter, left_height + right_height)

    # Return height of THIS subtree to the parent (+1 for the edge to parent)
    return 1 + max(left_height, right_height)

call height(root)
return diameter
```

3. Python Solution

```
from typing import Optional

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
```

```

self.left = left
self.right = right

```

```

class Solution:

```

```

    def diameterOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        # `diameter` stores the best (longest) path found so far, measured in EDGES.
        # We use a one-element list (or could use self.diameter) so the inner
        # function can mutate it - closures can't reassign an outer int directly.
        self.diameter = 0

        def height(node: Optional[TreeNode]) -> int:
            """
            Returns the height of the subtree rooted at `node`,
            where height = number of edges on the longest downward path.
            As a SIDE EFFECT, it updates self.diameter.
            """
            # Base case: an empty subtree contributes 0 edges of height.
            if node is None:
                return 0

            # Recursively get heights of left and right subtrees.
            left_height = height(node.left)
            right_height = height(node.right)

            # The longest path PASSING THROUGH this node connects the
            # deepest leaf on the left to the deepest leaf on the right.
            # Number of edges in that path = left_height + right_height.
            self.diameter = max(self.diameter, left_height + right_height)

            # Return this subtree's height to the caller.
            # +1 accounts for the edge connecting this node to its parent.
            return 1 + max(left_height, right_height)

        # Kick off the DFS; the return value (overall height) is ignored.
        height(root)
        return self.diameter

```

Comparison with the reference solution

The reference uses `return -1` for `None` and adds `2` when updating the answer. That's an alternative bookkeeping where `None` is treated as "height -1". My version uses the more intuitive convention: **height of `None` = 0** (no edges). Both are correct — they just shift the baseline by 1 consistently. I find the `0`-based version easier to reason about because `left_height + right_height` directly equals the edge count of the path.

4. Complexity Analysis

Metric	Complexity	Explanation
Time	$O(n)$	Each node is visited exactly once by the DFS; all work per node is $O(1)$.
Space	$O(h)$	Recursion call stack depth equals tree height <code>h</code> . Best case (balanced) $O(\log n)$, worst case (skewed/linked-list tree) $O(n)$.

Why not $O(n^2)$? A naive approach computes `height()` independently for every node when checking its diameter. That recomputes heights repeatedly, giving $O(n^2)$. By **returning the height while updating the answer**, we reuse computation and stay at $O(n)$.

5. Alternative Approaches

- Naive $O(n^2)$:** For each node, compute left/right heights separately and find the max `left + right`. Simple to write but recomputes heights — too slow for large trees.
- Iterative post-order DFS:** Replace recursion with an explicit stack, storing computed heights in a hash map (`node -> height`). Avoids deep recursion (useful if hitting Python's recursion limit on a 10^4 -node skewed tree). Still $O(n)$ time, $O(n)$ space.
- Return a tuple (`height`, `diameter`)** (like `my_solution.py`): Avoids the shared mutable state by passing both values up through the recursion. Functionally identical, arguably cleaner (no `self`/closure mutation), just slightly more verbose.

```
python def dfs(node): if not node: return 0, 0 # (height, diameter) lh, ld = dfs(node.left) rh, rd = dfs(node.right) height = 1 + max(lh, rh) diameter = max(ld, rd, lh + rh) # through-node vs. subtree best return height, diameter
```

Recommended: The single-pass DFS (my main solution) is the standard, optimal, and most readable approach for interviews.

Codex Solution

1. Intuition

The **diameter** is the longest path between any two nodes.

For any node, the longest path that **passes through that node** is:

```
height of left subtree + height of right subtree
```

Why?

Because a path going through a node can start from the deepest node in its left subtree, go up to the current node, then go down to the deepest node in its right subtree.

So, for every node, we want to know:

1. The height of its left subtree
2. The height of its right subtree
3. The best diameter seen so far

This is a classic **postorder DFS** pattern:

```
Solve left subtree
Solve right subtree
Use their results to solve current node
```

2. Approach

We use DFS to compute the height of each subtree.

Key definition

Let:

```
height(node) = number of nodes on the longest downward path from node
```

So:

```
height(None) = 0
height(leaf) = 1
```

At each node:

```
left_height = height(node.left)
right_height = height(node.right)

diameter_through_node = left_height + right_height
```

Why does `left_height + right_height` count edges?

Example:

```
  1
 / \
2   3
```

For node 1:

```
left_height = 1
right_height = 1
diameter = 1 + 1 = 2 edges
```

The path is:

```
2 -> 1 -> 3
```

which has 2 edges.

Pseudocode

```
answer = 0

function dfs(node):
    if node is null:
        return 0

    left_height = dfs(node.left)
    right_height = dfs(node.right)

    diameter_through_node = left_height + right_height
    answer = max(answer, diameter_through_node)

    return 1 + max(left_height, right_height)

dfs(root)

return answer
```

3. Python Solution

```
from typing import Optional

# Definition for a binary tree node.
# LeetCode usually provides this class automatically.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def diameterOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        # This variable stores the largest diameter found so far.
        # Diameter is measured in number of edges.
        self.answer = 0

    def dfs(self, node: Optional[TreeNode]) -> int:
        """
        Returns the height of the current subtree.

        Height here means:
        - number of nodes on the longest path from this node downward
        - height(None) = 0
        - height(leaf) = 1
        """

        # Empty subtree has height 0.
        if node is None:
            return 0

        # Recursively compute the height of the left subtree.
        left_height = self.dfs(node.left)

        # Recursively compute the height of the right subtree.
        right_height = self.dfs(node.right)

        # The longest path passing through this node uses:
        # deepest node in left subtree -> current node -> deepest node in right subtree
        #
        # Since left_height and right_height count nodes downward,
        # their sum gives the number of edges in this path.
        diameter_through_node = left_height + right_height
```

```

        # Update the global best diameter.
        self.answer = max(self.answer, diameter_through_node)

        # Return the height of this subtree to the parent.
        # We add 1 to include the current node.
        return 1 + max(left_height, right_height)

# Start DFS from the root.
dfs(root)

# Return the maximum diameter found.
return self.answer

```

4. Complexity Analysis

Let n be the number of nodes in the tree.

Time Complexity

$O(n)$

Each node is visited exactly once.

At each node, we do constant work:

```

compute left height
compute right height
update answer
return height

```

So the total time complexity is:

$O(n)$

Space Complexity

$O(h)$

where h is the height of the binary tree.

This space is used by the recursion call stack.

In the best case, for a balanced tree:

```

h = log n
space = O(log n)

```

In the worst case, for a completely skewed tree:

```

h = n
space = O(n)

```

So worst-case space complexity is:

$O(n)$

5. Alternative Approaches

1. Brute Force DFS

For every node:

1. Compute the height of its left subtree
2. Compute the height of its right subtree
3. Calculate the diameter through that node

This works, but height is recomputed many times.

Time complexity:

$O(n^2)$

in the worst case for a skewed tree.

2. Return Both Height and Diameter

Instead of using `self.answer`, each DFS call can return:

```
(height, diameter)
```

For each node:

```
height = 1 + max(left_height, right_height)
diameter = max(left_diameter, right_diameter, left_height + right_height)
```

This is also:

```
Time: O(n)
Space: O(h)
```

Example pattern:

```
def dfs(node):
    if node is None:
        return 0, 0

    left_height, left_diameter = dfs(node.left)
    right_height, right_diameter = dfs(node.right)

    height = 1 + max(left_height, right_height)
    diameter = max(
        left_diameter,
        right_diameter,
        left_height + right_height
    )

    return height, diameter
```

Both approaches are valid. The global variable version is slightly simpler.